Discrete Applied Mathematics 246 (2018) 140–153

Contents lists available at ScienceDirect

Discrete Applied Mathematics

journal homepage: www.elsevier.com/locate/dam

Prefix and suffix reversals on strings☆,☆☆

Guillaume Fertin, Loïc Jankowiak, Géraldine Jean *

Laboratoire des Sciences du Numérique de Nantes, UMR CNRS 6004, Université de Nantes, 2 rue de la Houssinière, 44322 Nantes Cedex 3, France

ARTICLE INFO

Article history:
Received 29 January 2016
Received in revised form 2 April 2017
Accepted 26 July 2017
Available online 6 September 2017

Keywords:
Prefix and suffix reversals
Strings
Algorithmic complexity
Approximation algorithms
Fixed-parameterized tractability

ABSTRACT

The Sorting by Prefix Reversals problem consists in sorting the elements of a given permutation π using a minimum number of prefix reversals, i.e. reversals that always involve the leftmost element of π. A natural extension of this problem is to consider strings, in which any letter may appear several times, rather than permutations. In strings, three different types of problems arise: *grouping* (given a string S, transform it so that all identical letters are consecutive), *sorting* (a constrained version of grouping, in which the target string must be lexicographically ordered) and *rearranging* (given two strings S and T, transform S into T). In this paper, we study these three problems, under an algorithmic viewpoint, in the setting where two operations, rather than one, are allowed: namely, *prefix and suffix* reversals — where a suffix reversal must always involve the rightmost element of the string. We first compare the "prefix reversals only" case to our case, before presenting a series of algorithmic results on these three problems concerning polynomiality, constant ratio approximation algorithms, NP-hardness and fixed-parameterized tractability. These results depend on the size k of the alphabet on which the strings are built, with a particular focus on small-sized alphabet instances (i.e., $k = O(1)$) and big-sized alphabet instances (i.e. $n - k = O(1)$, where n is the length of the input string(s)).

© 2017 Elsevier B.V. All rights reserved.

1. Introduction and notations

A usual task in comparative genomics consists in comparing pairs of genomes, in order to define how (dis)similar they are. Usually, a genome is modeled as a permutation π or a string S, and the pairwise comparison of genomes is done by computing a distance between them. Such a distance is generally defined based on so-called "rearrangements", which are large scale operations involving contiguous segments of π (resp. S). The most famous example of such rearrangements is *reversals* (see e.g. [3,9]), which take a contiguous segment of π (resp. S), reverse its order, and reincorporate this new segment at the same location. More generally, given a set $\mathcal{S}$ of possible rearrangements, the distance between two permutations π and π' (resp. two strings S and T) is defined as the minimum number of operations from $\mathcal{S}$ that are required to obtain π' starting from π (resp. to obtain T starting from S). Many studies are concerned with the special case where $|\mathcal{S}| = 1$, i.e. only one type of rearrangement is authorized. There is a very abundant literature on the subject, and we refer the reader to [7] for a survey on the topic. In this paper, our object of study is strings, a generalization of permutations, in which any letter may occur several times. The set of rearrangements we authorize here is composed of two operations: *prefix reversals*,

☆ Supported by GRIOTE project, funded by Région Pays de la Loire.

☆☆ A preliminary version of this work appeared in Proceedings of the 22nd edition of the International Symposium on String Processing and Information Retrieval (SPIRE 2015) Fertin et al. (2015) [6].

* Corresponding author.
E-mail addresses: guillaume.fertin@univ-nantes.fr (G. Fertin), loic.jankowiak@etu.univ-nantes.fr (L. Jankowiak), geraldine.jean@univ-nantes.fr (G. Jean).

http://dx.doi.org/10.1016/j.dam.2017.07.031
0166-218X/© 2017 Elsevier B.V. All rights reserved.

which necessarily involve the first letter of the string, and *suffix reversals*, which necessarily involve the last letter of the string. When dealing with strings, three types of problems are usually considered: *grouping* (given a string S, transform it so that all identical letters are consecutive), *sorting* (a constrained version of grouping, in which the target string must be lexicographically ordered) and *rearranging* (given two strings S and T, transform S into T). In this paper, we study these three problems, under an algorithmic viewpoint.

Terminology and basic properties.

Throughout the paper, and otherwise stated, any string S is a sequence of letters built on an alphabet Σ of cardinality k, is *fully k-ary* (i.e. each letter of Σ appears at least once in S), and is of length n. A letter l in S is said to be *lonely* whenever it appears only once in S, and *abundant* otherwise. When $k = n$, i.e. in the special case of permutations, the identity string is denoted I_n. Two strings S and T are said to be *compatible* if they contain the same multiset of letters. If S is the concatenation of two strings S_1 and S_2, it is denoted $S = S_1.S_2$, or S_1S_2 if clear from the context. If S is the concatenation of p occurrences of the same string X, this will be denoted $S = X^p$. For any $1 \leq i \leq j \leq n$, $S[i..j]$ denotes the substring of S starting at position i and ending at position j. A *block* in S is a maximal substring in which all letters are equal. Let $b(S)$ be the number of blocks in S. Given two compatible strings S and T, we let $b_{max}(S, T) = \max\{b(S), b(T)\}$, which we will write b_{max} in the rest of the paper if clear from the context. Given two integers $1 \leq i \leq j \leq n$, a *reversal* $\rho(i, j)$ of a string S is the operation that reverses $S[i..j]$ and reincorporates it at the same location, i.e. $S = s_1s_2 \dots s_{i-1}\underline{s_is_{i+1} \dots s_{j-1}s_j}s_{j+1} \dots s_n$ is transformed into $S' = s_1s_2 \dots s_{i-1}\underline{s_js_{j-1} \dots s_{i+1}s_i}s_{j+1} \dots s_n$. For readability, the substring $S[i..j]$ to be reversed will sometimes be underlined, as shown in the above example. If a reversal $\rho(1, n)$ is applied to a string S, we will say it is a *complete* reversal, and the resulting string will be written $\overline{S}$. A *prefix reversal* (resp. *suffix reversal*) is a reversal $\rho(1, j)$ with $1 \leq j \leq n$ (resp. $\rho(i, n)$ with $1 \leq i \leq n$), i.e. the first (resp. last) letter of S is involved in the reversal. We will often use the terms *p-reversal* (resp. *s-reversal*) to denote a reversal that is prefix (resp. suffix).

We say that a fully k-ary string S is *grouped* if it contains exactly k blocks, i.e. if, for any letter l in Σ, all occurrences of l are consecutive. We say that S is *sorted* if S is both grouped and lexicographically ordered. Given a string S, sort(S) will denote its *corresponding sorted string*, i.e. the unique string compatible with S that is lexicographically ordered. Given two compatible strings S and T, and given a set $\mathcal{S}$ of possible operations on strings, the *distance* between S and T is defined as the minimum number of operations from $\mathcal{S}$ that are required to obtain T, starting from S. In this paper, the two operations we allow are prefix reversals and suffix reversals. We are now ready to formally describe the three problems we are interested in, in their decision version.

Grouping Strings by Prefix and Suffix Reversals (GPSR)
Instance : A fully k-ary string S, an integer r.
Question : Is there a sequence of at most r p- and s-reversals that transforms S into a grouped string?

Sorting Strings by Prefix and Suffix Reversals (SPSR)
Instance : A fully k-ary string S, an integer r.
Question : Is there a sequence of at most r p- and s-reversals that transforms S into its corresponding sorted string sort(S)?

Rearranging Strings by Prefix and Suffix Reversals (RPSR)
Instance : Two compatible fully k-ary strings S and T, an integer r.
Question : Is there a sequence of at most r p- and s-reversals that transforms S into T?

Given a string S, the least integer r for which the answer to GPSR is positive is called the *grouping distance*, and is denoted $d^g_{PS}(S)$. Similarly, the sorting and rearranging distances are denoted $d^s_{PS}(S)$ and $d^r_{PS}(S, T)$. Note that when $k = n$ (i.e. strings are in fact permutations), the grouping problem is trivial (S is already grouped). Moreover, the sorting and rearranging problems become equivalent (up to a relabeling of the input permutations in the case of rearranging).

Note also that when dealing with the grouping (resp. sorting) problem, it is easy to show, by contradiction, that there always exists an optimal grouping (resp. sorting) algorithm for S that never cuts a block (otherwise, a more efficient algorithm would exist). Thus, when we look at GPSR or SPSR, the lengths of the blocks play no particular role, and it will thus be more convenient to assume that each block of S is of length 1. We will call such a string a *normalized* string. Consequently, any (prefix or suffix) reversal whose effect is to make two identical letters consecutive reduces the length of a normalized string by 1, and such a reversal is called a *1-flip*. A reversal that is not a 1-flip is called a *0-flip*.

Known results.

Comparing pairs of strings by means of their rearrangement distance has mostly been studied in the case where only one type of operation is allowed, such as reversals [5,14,1], transpositions [5,14] or translocations [4]. When prefix reversals only are allowed, the main results are the following: first, the grouping problem is polynomial when $k = 2$ and $k = 3$, and admits

Table 1
Algorithmic results obtained in this paper concerning problems GPSR and SPSR; $b(S)$ denotes the number of blocks of string S.

$\|\Sigma\|$	Grouping	Sorting
2	in P (Propositions 3 and 6)	
3	in P (Theorems 4 and 7)	
4	in P (Theorem 5)	PTAS (Theorem 8)
$O(1)$	PTAS (Theorem 6)	PTAS (Theorem 8)
$n-c$, with $c=O(1)$	in P (Proposition 4)	$O(1)$-approximation (Corollary 4)
n	in P (trivial)	NP-hard (Corollary 2)
any $2 \leq k \leq n$	2-approximation (Proposition 5)	
	FPT in $b(S)$ (Corollary 5)	FPT in $b(S)$ (Corollary 5)

Table 2
Algorithmic results obtained in this paper concerning problem RPSR; b_{max} denotes the maximum number of blocks between the two input strings S and T.

$\|\Sigma\|$	Rearranging
$c=O(1)$, with $c \geq 2$	NP-hard (Corollary 3)
$n-c$ with $c=O(1)$	$O(1)$-approximation (Proposition 9)
n	NP-hard (Theorem 9)
any $2 \leq k \leq n$	FPT in b_{max} (Theorem 10)

a PTAS for constant $k \geq 4$ [10]. Concerning sorting, the problem is polynomial for $k=2$ [4], and admits a PTAS for constant $k \geq 4$ [10]. Finally, concerning the rearrangement problem, it is NP-hard even for $k=2$ [10] (and for even more constrained such inputs [1]), it admits a PTAS for dense instances (which are instances for which the distance is cn with $c=O(1)$ [14]), and it is FPT in b_{max} [1]. When both prefix *and* suffix operations are permitted, recent works have either focused on the case of permutations [13,12], or on the case of transreversals on small-size alphabets ($k=2$ and $k=3$) [15].

Our results.

In this paper, we study the three problems GPSR, SPSR and RPSR, under an algorithmic viewpoint. The main algorithmic results we have obtained are summarized in Tables 1 and 2. This paper completes the preliminary version [6] by including all the proofs and original results concerning approximation algorithms for SPSR and RPSR problems.

The paper is organized as follows: in Section 2, we give elements of comparison, for the three considered problems, between the case where only prefix reversals are allowed and our case, i.e. when both prefix and suffix reversals are allowed. We then provide our algorithmic results for GPSR, SPSR and RPSR, respectively in Sections 3–5. Section 6 is our conclusion.

2. A detour by prefix reversals

In this section, our main goal is to compare the prefix distance to the prefix and suffix distance for grouping, sorting and rearranging. Let $d_P^g(S)$ (resp. $d_P^s(S)$, $d_P^r(S,T)$) denote the grouping (resp. sorting, rearranging) distance when only prefix reversals are allowed. First, note that for any string S, any grouping/sorting/rearranging algorithm that uses only p-reversals is also a valid algorithm using p- and s-reversals ; thus, trivially, for any string S, $d_{PS}^g(S) \leq d_P^g(S)$ and $d_{PS}^s(S) \leq d_P^s(S)$, while for any compatible strings S and T, $d_{PS}^r(S,T) \leq d_P^r(S,T)$. Notice also that any s-reversal $\rho(i,n)$ can be realized by at most 3 p-reversals (namely, $\rho(1,n)$, $\rho(1,n-i+1)$ and $\rho(1,n)$ again), which directly yields that for any string S, $d_P^g(S) \leq 3d_{PS}^g(S)$ and $d_P^s(S) \leq 3d_{PS}^s(S)$, while for any compatible strings S and T, $d_P^r(S,T) \leq 3d_{PS}^r(S,T)$. However, these upper bounds can be improved, as shown below.

Proposition 1. *For any compatible strings S and T, we have:*

(i) $d_{PS}^g(S) \leq d_P^g(S) \leq 2d_{PS}^g(S)$;
(ii) $d_{PS}^s(S) \leq d_P^s(S) \leq 2d_{PS}^s(S)+1$;
(iii) $d_{PS}^r(S,T) \leq d_P^r(S,T) \leq 2d_{PS}^r(S,T)+1$.

Proof. Let us first consider the grouping distance, and consider an optimal grouping algorithm $OPT_g(S)$ for S, which takes $d_{PS}^g(S)=p+s$ reversals, where p (resp. s) is the number of p- (resp. s-) reversal that is used during $OPT_g(S)$. Let S_g be the grouped string obtained this way. Now, consider the following alternative algorithm: completely reverse S (thus obtaining $\overline{S}$), and group $\overline{S}$ using algorithm $OPT_g(S)$, but in which every p- (resp. s-) reversal is replaced by an s- (resp. p-) reversal. Call this new algorithm $A_g(S)$. Notice that $A_g(S)$ groups S into $\overline{S_g}$, and that it uses $p+s+1$ reversals, except that here, p is the number of s-reversals, and s the number of p-reversals. Since any s-reversal can be simulated using 3 p-reversals, we have that $d_P^g(S) \leq p+3s$ from algorithm $OPT_g(S)$, and also $d_P^g(S) \leq 3p+s+1$ from algorithm $A_g(S)$, thus $d_P^g(S) \leq \min\{p+3s, 3p+s+1\}$.

For any positive integers a and b, $\min\{a, b\} \leq \lfloor \frac{a+b}{2} \rfloor$, thus $\min\{p + 3s, 3p + s + 1\} \leq 2p + 2s$, from which we conclude that $d^g_{PS}(S) \leq d^g_P(S) \leq 2d^g_{PS}(S)$.

The above proof can be adapted in order to cover the rearranging case. For this, it suffices to note that, starting from any optimal rearranging scenario $OPT_r(S, T)$ between two strings S and T, if we now apply $OPT_r(S, T)$ to $\overline{S}$ instead of S while swapping the p- and s-reversals, then we obtain $\overline{T}$. Thus, at the end of the process, one more complete reversal is enough to obtain T. If we simulate any s-reversal by 3 p-reversals in the two rearranging scenarios sketched above, we then have that $d^r_P(S) \leq \min\{p + 3s, 3p + s + 2\}$. By a similar analysis as for the grouping case, we have that $d^r_P(S) \leq 2d^r_{PS}(S) + 1$.

Concerning the SPSR problem, replacing T by sort(S) in the above argument, we get $d^s_P(S) \leq 2d^s_{PS}(S) + 1$. □

Another natural question that emerges is whether there exist examples of strings for which the prefix and prefix/suffix distances significantly differ. We answer this question positively for sorting in permutations (Theorem 1), and for rearranging in strings built on constant-size alphabets (Theorem 3).

Theorem 1. *For infinitely many values of n, there exists a permutation π of length n such that $d^s_P(\pi) - d^s_{PS}(\pi) = \Omega(n)$.*

Proof. The proof is inspired from proof of Theorem 2 in [8]. Take $n = 6q$ with even $q \geq 2$, and consider the following permutation $\pi = s_0\ s_1 \dots s_{q-1}$, where $s_i = 6i + 2\ 6i + 1\ 6i + 4\ 6i + 3\ 6i + 6\ 6i + 5$, $0 \leq i \leq q - 1$. Notice that here, $\Sigma = \{1, 2 \dots 6q\}$. We will show that $d^s_P(\pi) \geq \frac{n}{2} + \frac{n}{12}$, while $d^s_{PS}(\pi) \leq \frac{n}{2}$, which proves that $d^s_P(\pi) - d^s_{PS}(\pi) = \Omega(n)$.

In order to show that $d^s_{PS}(\pi) \leq \frac{n}{2}$, we claim that π can be sorted by repeating $\frac{n}{4}$ times the following two reversals: a p-reversal $\rho(1, n - 2)$ followed by an s-reversal $\rho(3, n)$. Indeed, it can be shown by induction that for any $1 \leq i \leq \frac{n}{4}$, after the ith s-reversal $\rho(3, n)$ has been applied, the permutation π' we obtain satisfies $\pi' = \pi_1 \cdot \pi_2$, where $\pi_1 = n - 4i + 1\ n - 4i + 2 \dots n - 1\ n$ contains the last $4i$ elements sorted in increasing order, and π_2 contains the first $n - 4i$ elements of the input permutation π. Thus, when $i = \frac{n}{4}$, the permutation is in fact I_n, and exactly $\frac{n}{2}$ reversals have been used.

Now let us prove $d^s_P(\pi) \geq \frac{n}{2} + \frac{n}{12}$. We say that a *breakpoint* occurs in π whenever $|\pi_{i+1} - \pi_i| \neq 1$, $1 \leq i \leq n$, where we artificially assume that $\pi_{n+1} = n + 1$. Consider an optimal sequence of p-reversals that sorts π, say $\mathcal{S}_\pi$, and let us distinguish in $\mathcal{S}_\pi$ between *efficient* p-reversals, which strictly reduce the number of breakpoints, and *wastes*, which do not. Hence, we have $d^s_P(\pi) = e + w$, where e (resp. w) denotes the number of efficient p-reversals (resp. wastes) in an optimal solution. Since π contains $\frac{n}{2}$ breakpoints, we necessarily have $e \geq \frac{n}{2}$. The rest of the proof is then dedicated to proving $w \geq \frac{n}{12}$. For each substring s_i, $0 \leq i \leq q - 1$, we say that a p-reversal is s_i-*breaking* if it is the last p-reversal of $\mathcal{S}_\pi$ before which s_i or $\overline{s_i}$ exists, and does not exist anymore afterwards. Let us sort the s_i-breaking p-reversals in their order of appearance, so as to obtain a sorted list of strictly positive integers $(a_1, a_2 \dots a_t)$, where for each $1 \leq j \leq t$, the a_j-th p-reversal of $\mathcal{S}_\pi$ is s_i-breaking for some $1 \leq i \leq q - 1$. We first note that this string contains exactly $t = q$ elements, since there are q strings to break, and at most one string can be broken by a p-reversal. We will now prove that $w \geq \frac{n}{12}$, by showing that it takes at least one waste to break two strings of the form s_i or $\overline{s_i}$. Consider any integer $j \leq q$, and let s be the string for which the a_j-th p-reversal is breaking. Let us first suppose $s = s_i$ (the case $s = \overline{s_i}$ is similar and leads to the same conclusion). Let π_0 be the permutation before the a_j-th p-reversal takes place. Then $\pi_0 = X.s_i.Y$. If X is empty, then $\pi_0 = s_i.Y$, and any breaking p-reversal for s_i is a waste. Otherwise, the only breaking p-reversal for s_i that is not a waste can occur if $X = (6i + 7).X'$, and leads to $\pi'_0 = 6i + 3\ 6i + 4\ 6i + 1\ 6i + 2.\overline{X'}.6i + 7\ 6i + 6\ 6i + 5.Y$. But then any p-reversal ρ that follows is necessarily a waste — however, ρ may also be breaking for another string $s_{i'}$. We thus have $w \geq \frac{q}{2} = \frac{n}{12}$, from which we conclude that $d^s_P(\pi) \geq \frac{n}{2} + \frac{n}{12}$. □

Since rearranging and sorting are equivalent when we deal with permutations, we have the following corollary.

Corollary 1. *For infinitely many values of n, there exist permutations π and π', of length n, such that $d^r_P(\pi, \pi') - d^r_{PS}(\pi, \pi') = \Omega(n)$.*

The following result, which will be helpful in the remainder of the paper, is an extension of [10] (Theorem 10) to p- and s-reversals and to constant-size alphabets.

Theorem 2. *For any pair of compatible strings S and T built on some alphabet Σ, and for any constant $k \geq 2$, there exists a pair of compatible strings S' and T' built on some alphabet Σ' satisfying $|\Sigma'| = k$, such that (i) $d^r_P(S', T') = d^r_P(S, T)$ and (ii) $d^r_{PS}(S', T') = d^r_{PS}(S, T)$.*

Proof. Suppose S and T are both of length n. The transformation from (S, T) to (S', T') we propose here is adapted from proof of Theorem 10 in [10], which concerned p-reversals and $k = 2$ only. We first note that a discussion which follows proof of Theorem 10 in [10] directly implies that, for p- and s-reversals, and for $k = 2$, the result holds, as Theorem 10 can be extended to other operations under certain conditions, which are fulfilled in our case. Indeed, for any compatible strings S and T of length n, we have (a) $d^r_{PS}(S, T) \leq 2n$, and (b) a p- (resp. an s-) reversal cuts a string in only one place. Here, we extend this result to *any constant* value of k. For this, we do the following: suppose wlog that S and T are built on $\Sigma = \{0, 1, 2 \dots |\Sigma| - 1\}$, and let $S = s_1 s_2 \dots s_n$ (resp. $T = t_1 t_2 \dots t_n$). Let k be any constant positive integer. We then let $S' = S'_1 S'_2 \dots S'_n$ and $T' = T'_1 T'_2 \dots T'_n$, where, for each $1 \leq i \leq n$, $S'_i = (X.0^{s_i+1}.\overline{X})^{2n+1}$ and $T'_i = (X.0^{t_i+1}.\overline{X})^{2n+1}$, with $X = 1\ 2\ 3 \dots k - 1$. Clearly, S' and T' are both built on $\Sigma' = \{0, 1, 2 \dots k - 1\}$, and it can be easily seen that $d^r_{PS}(S', T') \leq d^r_{PS}(S, T)$. Moreover, since any T'_i (resp. S'_i) is

reversal invariant, and because of (a) and (b) above, we conclude that $d^r_{PS}(S,T) \leq d^r_{PS}(S',T')$, because proof of Theorem 10 in [10] remains valid for this construction. The case where p-reversals only are allowed can be proved in a similar way, using the same construction as above. □

Putting together Corollary 1 and Theorem 2, we have the following result, which shows that rearranging by p- and s-reversals may be much more efficient than by p-reversals only, when dealing with constant-size alphabets.

Theorem 3. *Let $k \geq 2$ be any constant integer. For infinitely many values of n, there exist pairs (S,T) of compatible fully k-ary strings of length n such that $d^r_P(S,T) - d^r_{PS}(S,T) = \Omega(n^{\frac{1}{3}})$.*

Proof. Take two permutations π and π' of length ℓ that satisfy Corollary 1, i.e. $d^r_P(\pi,\pi') - d^r_{PS}(\pi,\pi') = \Omega(\ell)$. Now, use the transformation given in proof of Theorem 2, where π is transformed in S' and π' in T', and let $|S'| = |T'| = n$. We have that $n = \Theta(\ell^3)$ by construction, and since $d^r_P(S',T') = d^r_P(\pi,\pi')$ (resp. $d^r_{PS}(S',T') = d^r_{PS}(\pi,\pi')$) by Theorem 2, we conclude that $d^r_{PS}(S',T') - d^r_P(S',T') = \Omega(\ell) = \Omega(n^{\frac{1}{3}})$. □

3. Grouping strings by prefix and suffix reversals

This section describes our results concerning the GPSR problem. Note that all through this section, n will denote the length of the *normalized version* of the considered string.

Proposition 2. *For any $k \geq 3$, and for any fully k-ary string S of length n, $n-k \leq d^g_{PS}(S) \leq \min\{n-3, 2(n-k)\}$.*

Proof. Let S be a normalized fully k-ary string S of length n. First, it is easy to see that for any $k \geq 2$, $d^g_{PS}(S) \geq n-k$. Indeed, after S is grouped, its normalized string will be of length k. Since any reversal decreases the length of the (normalized) string by at most 1, we conclude that it takes at least $n-k$ reversals to group S.

Now, let us show that $d^g_{PS}(S) \leq \min\{n-3, 2(n-k)\}$ for any $k \geq 3$. Let us first show that $d^g_{PS}(S) \leq n-3$. For this, we apply the following algorithm, that we call $A^g_{PS}(S)$, which works in two phases: (i) while the last letter l of S is abundant, perform an s-reversal (which is in fact a 1-flip) to group l with another occurrence of it. We then end up with a string S', of length k', whose last letter is lonely. This last letter will then be ignored, and (ii) we will group the first $k'-1$ letters of S' using only p-reversals. The number of p- and s-reversals needed by $A^g_{PS}(S)$ is as follows: since $|S'| = k'$, we have performed $n-k'$ s-reversals during phase (i). Concerning phase (ii), by Lemma 2 of [10], we know that $(k'-1)-2$ p-reversals suffice to group this string. Thus, altogether, $A^g_{PS}(S)$ uses no more than $(n-k')+(k'-3) = n-3$ reversals.

Now let us prove that $d^g_{PS}(S) \leq 2(n-k)$. Let $A'^g_{PS}(S)$ be the following algorithm: while there exists an abundant letter l in the string, (i) if l appears at the first or last position, apply a 1-flip which groups two occurrences of l ; (ii) if not, apply a 0-flip to bring l either at the first or last position in the string. Clearly, when $A'^g_{PS}(S)$ stops, the string is grouped, and in its normalized version, is of length k. It is not hard to see that $A'^g_{PS}(S)$ uses exactly $(n-k)$ 1-flips. Moreover, since each 0-flip is necessarily followed by a 1-flip, and since the last reversal is necessarily a 1-flip, we conclude that the number of 0-flips cannot exceed the number of 1-flips, that is $n-k$. Thus, altogether, algorithm $A'^g_{PS}(S)$ uses no more than $2(n-k)$ reversals. □

Proposition 3. *For any fully binary string S of length n, $d^g_{PS}(S) = n-2$, and the* GPSR *problem is in P.*

Proof. Theorem 1 from [10] shows that $d^g_P(S) = n-2$ for any fully binary string S. Since $d^g_{PS}(S) \leq d^g_P(S)$, we also have $d^g_{PS}(S) \leq n-2$. Moreover, it can be seen that $d^g_{PS}(S) \geq n-k$ is also valid for $k=2$, as mentioned in proof of Proposition 2. We thus conclude that $d^g_{PS}(S) = n-2$. Now let us describe a grouping algorithm that uses $n-2$ reversals. First, it is easy to see that any normalized fully binary string S alternates 0s and 1s. Thus, while its length is at least 3, there always exists a 1-flip (that brings together two occurrences of the same letter, and reduces the string length by 1), and we apply such a reversal. This algorithm uses $(n-2)$ 1-flips. Because any normalized fully binary string is alternating, the 1-flip considered at each step of the algorithm to group S can always be a p-reversal of the two first elements that brings together the first and the third letters, and thus any 1-flip can be achieved in constant time. Our algorithm is thus in $O(n)$. □

Theorem 4. *For any fully ternary string S of length n, $d^g_{PS}(S) = n-3$ and the* GPSR *problem is in P.*

Proof. Setting $k=3$ in Proposition 2 suffices to get $d^g_{PS}(S) = n-3$ for any fully ternary string S of length n. Besides, proof of Proposition 2 describes an algorithm, $A^g_{PS}(S)$, that groups S using $(n-3)$ 1-flips. The time complexity of $A^g_{PS}(S)$ is $O(n^2)$, since the grouping distance is linear, and since choosing the 1-flip to perform, as well as performing it, can always be achieved in linear time. □

We know that $n-4 \leq d^g_{PS}(S) \leq n-3$ for any fully quaternary string S of length n (just invoke Proposition 2 with $k=4$). Our goal here is to find a polynomial-time algorithm that determines $d^g_{PS}(S)$ for any S, and that provides an optimal grouping scenario. Because the lower and upper bounds differ by one, determining $d^g_{PS}(S)$ consists in deciding whether S can be grouped using 1-flips only (leading to $d^g_{PS}(S) = n-4$), or whether one 0-flip is necessary (leading to $d^g_{PS}(S) = n-3$).

Table 3
Bad strings of length $n \geq 11$ for the GPSR problem with $k = 4$. For strings of type E, we have $a, b \geq 0$ and $a + b = p - 3$. For strings of types G and H, $c > 0$, $d \geq 0$ and $c + d = p - 2$.

$n = 2p + 1$ is odd		$n = 2p$ is even	
Type	Form	Type	Form
A	$x(\{y, z, t\}x)^p$	C	$(\{y, z, t\}x)^p$
B	$\{y, z, t\}(x\{y, z, t\})^p$	D	$(x\{y, z, t\})^p$
		E	$y(xz)^a xty(xz)^b xt$
		F	$x(yz)^{p-1}t$
		G	$x(yz)^c xy(zy)^d t$
		H	$t(yz)^d yx(zy)^c x$

Table 4
Every even-length string S' obtained from a bad odd-length string S by a 1-flip is bad.

S type $\xrightarrow{1-flip}$ S' type	S type $\xrightarrow{1-flip}$ S' type
A $\xrightarrow{p}$ C	B $\xrightarrow{p}$ D
A $\xrightarrow{s}$ D	B $\xrightarrow{s}$ C

Table 5
Every odd-length string S' obtained from a bad even-length string S by a 1-flip is bad. Note that bad strings of type F do not admit any 1-flip and bad strings of type G (resp. H) admit only a 1-flip which is a p-reversal (resp. an s-reversal).

S type $\xrightarrow{1-flip}$ S' type	S type $\xrightarrow{1-flip}$ S' type
C $\xrightarrow{p}$ A	E $\xrightarrow{p}$ B
C $\xrightarrow{s}$ B	E $\xrightarrow{s}$ B
D $\xrightarrow{p}$ B	G $\xrightarrow{p}$ B
D $\xrightarrow{s}$ A	H $\xrightarrow{s}$ B

Note that for $n \leq 11$, by exhaustive search, it is possible to determine which strings of length n need $n - 4$ reversals to be grouped, and which ones need $n - 3$ reversals. Moreover, for each of them, an optimal grouping scenario can be provided. We also determined different types of strings of length $n \geq 11$ that we call *bad* strings, which are listed in Table 3. Since relabeling a string S does not change its grouping distance, let $\Sigma = \{0, 1, 2, 3\}$ and let $x, y, z, t \in \Sigma$ with $x \neq y \neq z \neq t$.

First, we show that any bad string from Table 3 needs $n - 3$ reversals to be grouped.

Lemma 1. *For any bad fully quaternary string S of length $n \geq 11$, $d^g_{PS}(S) = n - 3$.*

Proof. The proof is by induction on n. The base case $n = 11$ can be easily verified. We now prove that if the statement holds for some $n \geq 11$, then it also holds for $n + 1$ by showing, by a case by case analysis, that any 1-flip on a bad string S of length $n + 1$ leads to a bad string S' of length n. To do so, we consider every type of bad string and every possible 1-flip on such type of string. Tables 4 and 5 summarize all the possible cases that occur when starting with a bad string S and applying a 1-flip. For each possible type of S, we consider all possible 1-flips (prefix and suffix), and show that we always end up with a (normalized) bad string S'. Table 4 is concerned with strings S of odd length, while Table 5 is concerned with strings S of even length. In these tables, each entry X $\xrightarrow{t}$ Y has the following meaning: starting from string S of type X, and applying a t-reversal (with $t \in \{p, s\}$), the string S' that is obtained is bad of type Y.

By induction hypothesis, we know that $d^g_{PS}(S') = n - 3$, thus $d^g_{PS}(S) = d^g_{PS}(S') + 1 = n - 2$, which completes the induction step. We thus conclude that $d^g_{PS}(S) = n - 3$ for any bad fully quaternary string S of length $n \geq 11$. □

We now want to show that no other string requires $n - 3$ reversals to be grouped. Let us call *good* any string of length $n \geq 11$ which is not bad. We have the following lemma.

Lemma 2. *For any good fully quaternary string S of length $n \geq 11$, $d^g_{PS}(S) = n - 4$.*

Proof. The proof is by induction on n. Again, for the base case, it can be easily verified that all good strings of length $n = 11$ can be grouped using $n - 4$ 1-flips. We now prove that if the statement holds for some $n \geq 11$, then it also holds for $n + 1$, by showing that if a 1-flip transforms a good string S of length $n + 1$ into a bad string S' (of length n), then there exists another 1-flip that transforms S into a good string S'' of length n.

The proof follows from a case by case analysis. We will list all good strings of length $n+1 \geq 12$ that could lead to a bad string of length n via a 1-flip. For each such string, we will show that at least one other 1-flip, leading to a good string, exists.

Case 1: $n+1$ is even, thus the bad string we obtain is of odd length.

(a) There is no good string leading to a bad string of type A via a 1-flip.

(b) First, we consider that the bad string of type B was obtained by a p-reversal. Hence, S is of the form $zx(\{y,z,t\}x)^a yz(x\{y,z,t\})^b$ with $a, b \geq 0$ and $a+b \geq 4$ since $n+1 \geq 12$. In all possible cases detailed below, it is always possible to apply a 1-flip on S that does not cut substring yz and thus leads to a good string:

- there exist at least three occurrences of z in S: we apply a prefix 1-flip in order to group the first z with another one without cutting substring yz.
- letter z appears only twice in S:
  - the last letter is z (i.e. $b=0$). Here, we can apply a suffix 1-flip that groups the two occurrences of z while preserving substring yz.
  - the last letter is y and there exist at least three occurrences of y in S (otherwise, S would be a bad string of type E). In that case, we can apply a suffix 1-flip that groups two occurrences of y without involving the one in substring yz.
  - the last letter is t and there exist at least two occurrences of t in S (otherwise, S would be a bad string of type G). In that case, we can apply a suffix 1-flip that groups two occurrences of t.

Note that a complete reversal on any bad string of type B leads to a string of same type. Thus, if we assume that the bad string of type B was obtained by a suffix 1-flip then we can symmetrically apply the same proof.

Case 2: $n+1$ is odd, thus the bad string we obtain is of even length.

(a) Any bad string of type C obtained by a 1-flip from a good string has necessarily been obtained by a p-reversal, since any suffix 1-flip that leads to a bad string of type C comes from a bad string of type B. Thus, we can consider that S is of the form $zx(\{y,z,t\}x)^a yzx(\{y,z,t\}x)^b$ with $a, b \geq 0$ and $a+b \geq 4$ since $n+1 \geq 12$. In all possible cases detailed below, it is always possible to apply a 1-flip that leads to a good string:

- there exist at least three occurrences of z in S. In that case, we can apply a prefix 1-flip that groups two occurrences of z without involving the one in substring yz.
- letter z appears only twice in S, and so y or t (or both) is abundant:
  - letter y appears at least twice in S and we can always apply the underlined suffix 1-flip that preserves substring yz:
    $S = zx(\{y,t\}x)^a yzx(\{y,t\}x)^b$, $S'' = zx(\{y,t\}x)^{a+b}zy$.
  - letter t appears at least twice in S. There always exists a suffix 1-flip that groups two occurrences of x, and brings an occurrence of t to the last position without cutting substring yz, leading to a good string of the form $S'' = zx(\{y,t\}x)^a yzx(\{y,t\}x)^{b-1}t$ or $S'' = zx(\{y,t\}x)^{a'}zyx(\{y,t\}x)^{b'}t$ with $a', b' \geq 0$ and $a'+b' = a+b-1$.

(b) Any bad string of type D is obtained by a complete reversal of a string of type C, thus a similar analysis applies.

(c) First suppose that bad string S' of type E has been obtained by a prefix 1-flip. In that case, there are six possibilities for S and, in all cases, a good string S'' can be obtained by applying the underlined 1-flip on S:

- $S = \underline{tx(zx)^b}ytx(zx)^a yt$ with $a, b \geq 0$ and $a+b \geq 3$
- $S = \underline{tx(zx)^a yty(xz)^b xt}$ with $a, b \geq 0$ and $a+b \geq 3$
- $S = \underline{x(zx)^{a'}y(xz)^{a''}xty(xz)^b xt}$ with $a', a'', b \geq 0$ and $a'+a''+b \geq 3$
- $S = \underline{x(zx)^{b'}ytx(zx)^a y(xz)^{b''}xt}$ with $a, b', b'' \geq 0$ and $a+b'+b'' \geq 3$
- $S = (zx)^{b'}ytx\underline{(zx)^a yz(xz)^{b''}xt}$ with $b' > 0$, $a, b'' \geq 0$ and $a+b'+b'' \geq 3$
- $S = (zx)^{a'}yz(xz)^{a''}xt\underline{y(xz)^b xt}$ with $a' > 0$, $a'', b \geq 0$, $a'+a''+b \geq 3$.

If S' has been obtained by an s-reversal, we obtain the same six cases by symmetry, since (completely) reversing a bad string of type E leads to a bad string of same type.

(d) Bad strings of type F have the form $S' = x(yz)^{p-1}t$ with $p \geq 6$ since $n+1 \geq 12$. A bad string of type F is identical to its respective string obtained from a complete reversal, up to relabeling. Then, we assume wlog that the bad string has been obtained by a prefix 1-flip. In that case, there are three possibilities for S and, in all cases, a good string S'' can be obtained via the underlined 1-flip:

- $S = y(zy)^a x(yz)^b t$ with $a \geq 0$, $b > 0$ and $a+b \geq 5$ since $n+1 \geq 12$:
  - case $a \geq 1$: $S = \underline{yz}y(zy)^{a-1}x(yz)^b t$
  - case $a = 0$ and $b \geq 5$: $S = \underline{yxyz}(yz)^{b-1}t$
- $S = (zy)^c xz(yz)^d t$ with $c > 0$, $d \geq 0$ and $c+d \geq 5$ since $n+1 \geq 12$:

  - case $c \geq 2$: $S = \underline{zy}(zy)^{c-1}xz(yz)^{d}t$
  - case $c = 1$ and $d \geq 4$: $S = \underline{zyxzyz}(yz)^{d-1}t$
- $S = t\underline{(zy)^{p-1}xt}$.

(e) There are five types of good strings which can lead to a bad string of type G after a prefix 1-flip. In all cases, a good string S'' can be obtained via the underlined 1-flip:

- $S = \underline{y(zy)^{a'}x(yz)^{a''}xy}(zy)^{b}t$ with $a', a'', b > 0$ and $a' + a'' + b \geq 4$
- $S = (zy)^{a'}xz(yz)^{a''}xy(zy)^{b}t$ with $a' > 0, a'', b \geq 0$ and $a' + a'' + b \geq 4$:
  - case $a' \geq 2$: $S = \underline{zy}(zy)^{a'-1}xz(yz)^{a''}xy(zy)^{b}t$
  - case $a' = 1, a'' \geq 1$: $S = \underline{zyxzyz}(yz)^{a''-1}xy(zy)^{b}t$
  - case $a' = 1, a'' = 0, b \geq 1$: $S = \underline{zyxzxy}(zy)^{b}t$
- $S = y(zy)^{b'}xzy(zy)^{a-1}xy(zy)^{b''}t$ with $a > 0, b', b'' \geq 0, a + b' + b'' \geq 4$
- $S = \underline{(zy)^{b'}x}(zy)^{a}x(zy)^{b''}t$ with $a, b', b'' > 0$ and $a + b' + b'' \geq 5$
- $S = t\underline{(yz)^{b}yx}(zy)^{a}\underline{xt}$ with $a > 0, b \geq 0$ and $a + b \geq 4$.

There exist six good strings that lead to S' after a suffix 1-flip. In all cases, a good string S'' can be obtained via the underlined 1-flip:

- $S = \underline{xt(yz)^{b}yx}(zy)^{a}x$ with $a > 0, b \geq 0$ and $a + b \geq 4$
- $S = \underline{x(yz)^{a}xt(yz)^{b}yx}$ with $a > 0, b \geq 0$ and $a + b \geq 4$
- $S = x(yz)^{a-1}\underline{yzxy(zy)^{b'}t(yz)^{b''}y}$ with $a > 0, b', b'' \geq 0, a + b' + b'' \geq 4$
- $S = x(yz)^{a}\underline{xy(zy)^{b'}zt(yz)^{b''}}$ with $b' \geq 0, a, b'' > 0$ and $a + b' + b'' \geq 4$
- $S = \underline{x(yz)^{a'}t(yz)^{b}yx}(zy)^{a''}z$ with $a' > 0, a'', b \geq 0$ and $a' + a'' + b \geq 4$
- $S = \underline{x(yz)^{a'}yt(yz)^{b}yx}(zy)^{a''}$, with $a', b \geq 0, a'' > 0$ and $a' + a'' + b \geq 4$.

(f) Any bad string of type H is obtained by a complete reversal of a string of type G, thus a similar analysis applies.

We have shown that it is always possible to apply a 1-flip to a good string S of length $n + 1$ that leads to another good string S'' of length n. By induction hypothesis, we know that $d^g_{PS}(S'') = n - 4$, thus $d^g_{PS}(S) = d^g_{PS}(S'') + 1 = n - 3$, which completes the induction step.

We thus conclude that any good string of length $n \geq 11$ has grouping distance equal to $n - 4$ since it can be grouped by 1-flips only. □

Putting Lemmas 1 and 2 and our exhaustive search for $n \leq 11$ together leads to the following theorem.

Theorem 5. *Let S be a fully quaternary string of length $n \geq 11$. If S is bad, then $d^g_{PS}(S) = n - 3$, while if S is good, then $d^g_{PS}(S) = n - 4$. Hence, the* GPSR *problem is in P for $k = 4$.*

Proof. Let S be a fully quaternary string of length $n \geq 11$. Determining whether S belongs to one of the types of bad strings given in Table 3 can be done in $O(n)$. If S is bad, apply algorithm $A^g_{PS}(S)$ described in proof of Proposition 2 to optimally group it in polynomial time. Otherwise, apply a series of 1-flips that only leads to good strings (which is always possible by Lemma 2) until we obtain a grouped string of length $n = 11$. From then on, follow an optimal grouping scenario provided by our exhaustive search for $n \leq 11$. This algorithm can be achieved in $O(n^3)$ since the grouping distance, the number of possible 1-flips at each iteration, and testing at each iteration whether the obtained string is good are all in $O(n)$. □

We end this section with three algorithmic results concerning more general values of k: when $k = O(1)$ (Theorem 6), when $n - k = O(1)$ (Proposition 4), and for any value of k (Proposition 5).

Theorem 6. *For any constant integer $k \geq 5$, and for any fully k-ary string, the* GPSR *problem admits a PTAS.*

Proof. Let S be a fully k-ary string, where $k \geq 5$ is a constant. For any fixed $\epsilon > 0$, we set $N = (k - 3)/\epsilon + k$, and we distinguish two cases. First, suppose $n \geq N$. In proof of Proposition 2, an algorithm $A^g_{PS}(S)$ that groups S using at most $r(S) = n - 3$ reversals is given and we also know that $d^g_{PS}(S) \geq n - k$ (i). Since $n \geq N$, we have that $d^g_{PS}(S) \geq N - k$, that is $d^g_{PS}(S) \geq \frac{k-3}{\epsilon} + k - k$, from which we conclude that $\epsilon d^g_{PS}(S) \geq k - 3$ (ii). Adding inequalities (i) and (ii) yields $(1 + \epsilon)d^g_{PS}(S) \geq n - 3$, i.e. $r(S) \leq (1 + \epsilon)d^g_{PS}(S)$, which shows that algorithm A^g_{PS} is a PTAS. Second, suppose that $N > n$. In that case, since ϵ and k are assumed to be constant, so is N. Hence, applying an exhaustive search from S will yield the exact value of $d^g_{PS}(S)$. Let us call this algorithm G^g_{PS}. Note that, starting from any string of length n, there are at most $2n - 3$ different possible reversals: $n - 2$ of each kind (prefix or suffix), and reversing the whole string. So, at each iteration in G^g_{PS}, there are at most $2n - 3 < 2N - 3$ possible reversals. Besides, since $d^g_{PS}(S) \leq n - 3$ (by Proposition 2), we conclude that G^g_{PS} can be achieved in $O((2n - 3)^{n-3}) = O((2N - 3)^{N-3}) = O(1)$, which concludes the proof. □

Proposition 4. *The* GPSR *problem is in P for any fully k-ary string of length n such that* $n - k = O(1)$.

Proof. Suppose $k = n - c$ with $c = O(1)$. By Proposition 2, we know that $d^g_{PS}(S) \leq \min\{n-3, 2(n-k)\}$, that is $d^g_{PS}(S) \leq 2c$. Thus, consider the following algorithm: starting from S, branch on all the possible p- and s-reversals, up to a distance of $2c$, while ceasing to explore further a branch for which the string is grouped. Then, return the smallest number of reversals that has been found, for reaching a grouped string, starting from S. Thus, exploring all the possible reversals up to a distance of $2c$ cannot create more than $(2n-3)^{2c}$ strings, since there are at most $2n-3$ different possible reversals starting from any string, as explained in proof of Theorem 6. By Proposition 2, we know that the optimal solution lies within this search space. Besides, since $c = O(1)$, the number of generated strings is polynomial, and consequently so is our algorithm. □

Proposition 5. *For any* $2 \leq k \leq n$, GPSR *is 2-approximable.*

Proof. Proof of Proposition 2 presents an algorithm A'^g_{PS} that uses at most $2(n-k)$ reversals, while Proposition 2 also yields a lower bound of $n-k$ reversals. Thus A'^g_{PS} is a 2-approximation algorithm for GPSR. It runs in $O(n^2)$ since the distance is in $O(n)$, and since each reversal can be decided and applied in linear time. □

4. Sorting strings by prefix and suffix reversals

In this section, we describe our results concerning the SPSR problem. Note that, as in the previous section, n denotes here the length of the considered normalized string. Note also that for any fully k-ary string S of length n, $d^s_{PS}(S) \geq d^g_{PS}(S)$. Since $d^g_{PS}(S) \geq n-k$ by Proposition 2, we obtain the following result.

Observation 1. *For any* $k \geq 2$, *and for any fully k-ary string S of length n,* $d^s_{PS}(S) \geq n-k$.

Using Observation 1, we get the following result.

Proposition 6. *For any fully binary string S of length n,* $d^s_{PS}(S) = n-2$, *and the* SPSR *problem is in P.*

Proof. Let S be any fully binary string. First, setting $k=2$ in Observation 1 yields $d^s_{PS}(S) \geq n-2$. It remains to show that $d^s_{PS}(S) \leq n-2$. For this, suppose $\Sigma = \{0,1\}$, and note that in any normalized fully binary string, 0s and 1s alternate. Thus, while its length is at least 3, there exists a 1-flip, i.e. a reversal that brings together two occurrences of the same letter, and reduces the normalized string length by 1. Apply such an algorithm till the normalized string S' is of length 3 (note that $(n-3)$ 1-flips have thus been used). Then there are two cases: either $S' = 010$, and in that case we apply the suffix 1-flip $\rho(2,3)$; or $S' = 101$, and in that case we apply the prefix 1-flip $\rho(1,2)$. In both cases, we obtain the sorted string 01. Altogether, we are able to sort S in $n-2$ reversals, hence the result. □

Proposition 7. *For any fully ternary string S of length* $n \geq 4$, $d^s_{PS}(S) \leq n-2$.

Proof. Assume $\Sigma = \{0,1,2\}$. By Proposition 2, since $k=3$, we know that $d^g_{PS}(S) \leq n-3$. Suppose we have grouped S in $n-3$ reversals, and let us list the possible strings S' we end up with:

- $S' = 012$, thus S' is already sorted.
- $S' = 210$. In that case, $\rho(1,3)$ sorts S'.
- $S' = 021$ or $S' = 102$. It can be easily seen that one more reversal ($\rho(2,3)$ in the first case, $\rho(1,2)$ in the second case) suffices to sort S'.
- $S' = 120$. In that case, S' has been obtained from 4 possible strings S'': 2120, 0210, 1202 or 1021. However, it can be seen that each of these four strings can be sorted in at most 2 reversals. Thus $n-2$ reversals are sufficient for sorting S: $n-4$ from S to S'', and 2 from S'' to 012.
- $S' = 201$. Similarly to the previous case, S' has been obtained from 4 possible strings S'', namely 0201, 1021, 2010 or 2102. As in the previous case, any such string S'' can be sorted in at most 2 reversals, which shows that $n-2$ reversals are sufficient to sort S.

In all cases, we see that $n-2$ reversals at most are needed to sort S, hence the result. □

Observation 1 and Proposition 7 show that when $k=3$, $n-3 \leq d^s_{PS}(S) \leq n-2$ for any fully ternary string S. In the following, we will show that there exists a polynomial-time algorithm that determines $d^s_{PS}(S)$ for any fully ternary string S, and provides an optimal sorting scenario. Because the lower and upper bounds differ by one, determining $d^s_{PS}(S)$ consists in deciding whether S can be sorted using 1-flips only (leading to $d^s_{PS}(S) = n-3$), or whether one 0-flip is necessary (leading to $d^s_{PS}(S) = n-2$).

Note that for $n \leq 9$, by exhaustive search, it is possible to determine which strings of length n need $n-3$ reversals to be sorted, and which ones need $n-2$ reversals. Moreover, for each of them, an optimal sorting scenario can be provided. Now suppose $n \geq 9$ and $\Sigma = \{0,1,2\}$, and let Table 6 describe a set of strings that we call *bad* strings.

Our goal is to show that, for any $n \geq 9$, the bad strings we have listed are the only ones having sorting distance $n-2$. We start with proving that any bad string has sorting distance $n-2$.

Table 6
Bad strings of length $n \geq 9$ for the SPSR problem with $k = 3$.

$n = 2p + 1$ is odd		$n = 2p$ is even			
Type	Form	Type	Form	Type	Form
A	$\{1, 2\}(0\{1, 2\})^p$	G	$(0\{1, 2\})^p$	L	$1(02)^{p-1}1$
B	$\{0, 1\}(2\{0, 1\})^p$	H	$(2\{0, 1\})^p$	M	$1(20)^{p-1}1$
C	$(0\{1, 2\})^p0$	I	$(\{0, 1\}2)^p$	N	$2(01)^{p-1}2$
D	$2(\{0, 1\}2)^p$	J	$(\{1, 2\}0)^p$	O	$2(10)^{p-1}1$
E	$2(10)^p$	K	$0(12)^{p-1}0$	P	$(12)^{p-1}10$
F	$(21)^p0$				

Table 7
Every even-length string S' obtained from a bad odd-length string S by a 1-flip is bad.

S type $\xrightarrow{1-flip}$ S' type	S type $\xrightarrow{1-flip}$ S' type
A $\xrightarrow{p}$ G	C $\xrightarrow{s}$ G
A $\xrightarrow{s}$ J	D $\xrightarrow{p}$ I
B $\xrightarrow{p}$ H	D $\xrightarrow{s}$ H
B $\xrightarrow{s}$ I	E $\xrightarrow{s}$ O
C $\xrightarrow{p}$ J	F $\xrightarrow{p}$ P

Table 8
Every odd-length string S' obtained from a bad even-length string S by a 1-flip is bad.

S type $\xrightarrow{1-flip}$ S' type	S type $\xrightarrow{1-flip}$ S' type	S type $\xrightarrow{1-flip}$ S' type
G $\xrightarrow{p}$ A	J $\xrightarrow{p}$ C	M $\xrightarrow{p}$ B
G $\xrightarrow{s}$ C	J $\xrightarrow{s}$ A	M $\xrightarrow{s}$ A
H $\xrightarrow{p}$ B	K $\xrightarrow{p}$ F	N $\xrightarrow{p}$ A
H $\xrightarrow{s}$ D	K $\xrightarrow{s}$ B	N $\xrightarrow{s}$ E
I $\xrightarrow{p}$ D	L $\xrightarrow{p}$ A	O $\xrightarrow{s}$ E
I $\xrightarrow{s}$ B	L $\xrightarrow{s}$ B	P $\xrightarrow{p}$ F

Lemma 3. *For any bad fully ternary string S of length $n \geq 9$, $d^s_{PS}(S) = n - 2$.*

Proof. The proof is by induction on n. First, for $n = 9$, it is easy to verify that the result holds. We now prove that if the statement holds for bad strings of length $n \geq 9$, then it also holds for bad strings of length $n + 1$. For this, we show, by a case by case analysis, that any 1-flip on a bad string S of length $n + 1$ leads to a bad string S' of length n. To do so, all types of bad strings and all possible 1-flips on such types of string are considered. Tables 7 and 8 summarize all the possible cases that occur when starting with a bad string S and applying a 1-flip. For each possible type of S, we consider all possible 1-flips (prefix and suffix), and show that we always end up with a (normalized) bad string S', whose type is listed in the tables. Table 7 is concerned with strings S of odd length, while Table 8 is concerned with strings S of even length. In these tables, each entry X $\xrightarrow{t}$ Y has the following meaning: starting from string S of type X, and applying a t-reversal (with $t \in \{p, s\}$), the string S' that is obtained is bad of type Y. By induction hypothesis, we know that $d^s_{PS}(S') = n - 2$, thus $d^s_{PS}(S) = d^s_{PS}(S') + 1 = n - 1$, which completes the induction step. We thus conclude that $d^s_{PS}(S) = n - 2$ for any bad fully ternary string S of length $n \geq 9$. □

It now remains to show that no other string requires $n - 2$ reversals to be sorted. Let us call *good* any string which is not bad. We have the following lemma.

Lemma 4. *For any good fully ternary string S of length $n \geq 9$, $d^s_{PS}(S) = n - 3$.*

Proof. The proof is by induction on n. Again, the result can easily be verified for good strings of length $n = 9$, which constitutes our base case. We now prove that if the statement holds for good strings of length $n \geq 9$, it also holds for good strings of length $n + 1$. We do this by showing that there always exists a 1-flip that transforms a good string S of length $n + 1$ into a good string of length n. In other words, if a 1-flip transforms S into a bad string S', then there exists another 1-flip that transforms S into a good string S''. We list all the good strings of length $n + 1 \geq 10$ that could lead to a bad string of length n via a 1-flip. For each such string, we show that at least one other 1-flip, leading to a good string, exists.

Case 1: $n + 1$ is even, thus the bad string we obtain is of odd length.

(a) All good strings leading to a bad string of type A (resp. type B) are of the form $(x0)(\{1,2\}0)^a yx(0\{1,2\})^b$ (resp. $(x2)(\{0,1\}2)^a yx(2\{0,1\})^b$), where $x \neq y \in \{1,2\}$ (resp. $x \neq y \in \{0,1\}$), $a, b \geq 0$, and $a + b \geq 3$ (since $n + 1 \geq 10$). It can be seen that it is always possible to apply a 1-flip that does not cut substring yx, and thus leads to a good string.
(b) There is no good string leading to a bad string of type C or D via a 1-flip.
(c) If S leads to a bad string of type E, then necessarily $S = (10)^a 12(10)^b$ or $S = (01)^a 20(10)^b$ with $a \geq 1$, $b \geq 0$ and $a + b \geq 4$. It can be seen that there exists a 1-flip which does not cut substring 12 (resp. 20), and thus leads to a good string.
(d) If S leads to a bad string of type F, then necessarily $S = (21)^a 01(21)^b$ or $S = (21)^a 20(12)^b$ with $a \geq 1$, $b \geq 0$ and $a + b \geq 4$. It can be seen that there exists a 1-flip which does not cut substring 01 (resp. 20), and thus leads to a good string.

Case 2: $n + 1$ is odd, thus the bad string we obtain is of even length.

(a) There is no good string leading to a bad string of type G, H, I or J via a 1-flip.
(b) There are four types of good strings which may lead to a bad string of type K after a 1-flip, and they are of the following form:

(i) $(12)^a 10(12)^b 0$ with $a \geq 0, b \geq 1$ and $a + b \geq 3$
(ii) $(21)^a 02(12)^b 0$ with $a \geq 1, b \geq 0$ and $a + b \geq 3$
(iii) $0(12)^a 10(21)^b$ with $a \geq 0, b \geq 1$ and $a + b \geq 3$
(iv) $0(12)^a 02(12)^b$ with $a \geq 1, b \geq 0$ and $a + b \geq 3$.

It can be seen that there always exists a 1-flip that does not cut substring 10 (cases (i) and (iii)) or substring 02 (cases (ii) and (iv)), and thus leads to a good string.
(c) Types L, M and N are similar to type K above: the general form of the string is the same, only the letters change. Thus, we can use similar arguments as for case (b) above, and conclude that there always exists an alternate 1-flip that leads to a good string.
(d) There are three types of good strings which may lead to a bad string of type O after a 1-flip, and they are of the following form:

(i) $(10)^a 12(10)^b 1$ with $a \geq 0, b \geq 0$ and $a + b \geq 3$
(ii) $(01)^a 20(10)^b 1$ with $a \geq 1, b \geq 0$ and $a + b \geq 3$
(iii) $2(10)^a 12$ with $a \geq 3$.

In case (i) (resp. (ii)), there exists a 1-flip that does not cut substring 12 (resp. 20), and thus leads to a good string. In case (iii), applying the only possible prefix 1-flip leads to a good string.
(e) There are three types of good strings which may lead to a bad string of type P after a 1-flip, and they are of the following form:

(i) $(12)^a 101(21)^b$ with $a \geq 0, b \geq 0$ and $a + b \geq 3$
(ii) $(12)^a 012(12)^b$ with $a \geq 1, b \geq 0$ and $a + b \geq 3$
(iii) $0(12)^a 10$ with $a \geq 3$

In case (i) (resp. (ii)), there exists a 1-flip that does not cut substring 10 (resp. 01), and thus leads to a good string. In case (iii), applying the only possible suffix 1-flip leads to a good string.

We have shown that it is always possible to apply a 1-flip to a good string S of length $n+1$ that transforms it into another good string S'' of length n. By induction hypothesis, we know that $d^s_{PS}(S'') = n - 3$, thus $d^s_{PS}(S) = d^s_{PS}(S'') + 1 = n - 2$, which completes the induction step. We thus conclude, by induction, that any good string has sorting distance equal to $n - 3$ since it can be sorted by 1-flips only. □

Putting Lemmas 3 and 4 and our exhaustive search for $n \leq 9$ together leads to the following theorem.

Theorem 7. *Let S be a fully ternary string of length $n \geq 9$. If S is bad, then $d^s_{PS}(S) = n-2$, while if S is good, then $d^s_{PS}(S) = n-3$. Hence, the* SPSR *problem is in P for $k = 3$.*

Proof. Given a fully ternary string S of length $n \geq 9$, it can be decided in $O(n)$ whether S is good or bad. Moreover, a sorting scenario achieving the optimal distance can always be found in $O(n^3)$: if S is bad, iteratively operate arbitrary 1-flips, until the string becomes of length $n = 9$. From then on, follow an optimal sorting scenario (that necessarily uses a 0-flip). This algorithm is in $O(n^2)$, since the distance is in $O(n)$, and since applying a 1-flip can be done in linear time. If S is good, at each iteration, choose a 1-flip that leads to a good string, until the string is of length $n = 9$. From then on, follow an optimal sorting scenario (that always uses 1-flips). The latter algorithm runs in $O(n^3)$ time, since at each of the $O(n)$ iterations, a choice has to be made among $O(n)$ possible 1-flips, and $O(n)$ time is used to check whether the obtained string is good. □

Proposition 8. *For any fully quaternary string S of length $n \geq 4$, $d^s_{PS}(S) \leq n - 1$.*

Proof. Let S be a fully quaternary string of length n (thus $n \geq 4$), with $\Sigma = \{0, 1, 2, 3\}$. We first note that when $n = 4$, $d^s_{PS}(S) \leq 3$, which satisfies the proposition. More precisely, the distance is at most 2, except for the following four strings $S^*_1 = 0213$, $S^*_2 = 1302$, $S^*_3 = 2031$ and $S^*_4 = 3120$, for which the distance is equal to 3. Now, suppose $n \geq 5$, and let S be a fully quaternary string of length n. We distinguish two cases: first, when $S[1]$ and $S[n]$ are both lonely; second, when at least one of $S[1]$ and $S[n]$ is abundant in S. We begin by the following claim.

Claim. For any $n \geq 4$, if $S[1]$ and $S[n]$ are both lonely in S, then $d^s_{PS}(S) \leq n - 1$.

Proof of Claim. We will in fact show that it is always possible to group S, using $n-3$ reversals, in such a way that it reaches a string T of length 4 satisfying $T \neq S^*_i$ for any $1 \leq i \leq 4$. Thus T can be sorted in at most 2 reversals, which proves the result. Wlog, let $S[1] = x$, and $S[n] = y$, with $x \neq y$ and $x, y \in \Sigma$. Let $S' = S[2..(n-1)]$. Since $|\Sigma| = 4$, we know that S' is a fully binary string. Wlog, let z, t be the letters used in S'. First, note that if $\{x, y\} \neq \{0, 2\}$ (resp. $\{x, y\} \neq \{1, 3\}$), then one s-reversal can be used to group x and y at the beginning of the string, and $(n-4)$ s-reversals are sufficient to group $S[3..n]$. We thus end up with $T = xyzt$ or $T = xytz$, which can be sorted with at most 2 reversals, since $T \neq S^*_i$ for any $i \in [1; 4]$. We are left with the cases $\{x, y\} = \{0, 2\}$ or $\{x, y\} = \{1, 3\}$. Wlog, let S' start with z. Thus, there are two cases to consider for S', depending on its last letter:

1. $S' = ztzt \dots ztzt$, that is $S = xztzt \dots ztzty$. In that case, we apply a p-reversal of length $n-2$, to obtain $ztzt \dots tzxty$. Now the first $(n-3)$ elements of the string yield a binary substring, that can be grouped to obtain tz in $(n-5)$ p-reversals. We thus end up with the string $tzxty$, which reduces to $T = xzty$ after one more p-reversal. But since $\{x, y\} = \{0, 2\}$ or $\{x, y\} = \{1, 3\}$, we know that $T \neq S^*_i$ for any $i \in [1; 4]$.
2. $S' = ztzt \dots ztz$, that is $S = xztzt \dots ztztzy$. Note that in that case, n is odd. If $n = 5$, i.e. $S = xztzy$, then in two p-reversals, we can obtain $T = txzy$, which is different from any S^*_i. Now if $n \geq 7$, then applying an s-reversal of length 2 followed by an s-reversal of length $n-2$ yields a string $U = xzyU'$, where $U' = tztz \dots tzt$ is of odd length $n-4 \geq 3$. Now, with $(n-6)$ s-reversals, it is possible to group U' into tz, and we end up with string $xzytz$. One last s-reversal then gives us $T = xzty$, and T is none of the S^*_i because we supposed $\{x, y\} = \{0, 2\}$ or $\{x, y\} = \{1, 3\}$.

The above claim considers the case where both $S[1]$ and $S[n]$ are lonely, which leaves us with the case where at least one of $S[1]$ and $S[n]$ is abundant in S — call this an *abundant* string. In that case, we will prove that $d^s_{PS}(S) \leq n-1$ by induction on n. When $n = 4$, no abundant string exists, thus we consider $n = 5$ as our base case. When $n = 5$, starting from an abundant string S, we know that there exists a 1-flip leading to a string of length 4, for which we know (by the above discussion) that it can be sorted in at most 3 reversals, thus $d^s_{PS}(S) \leq 4$. Now suppose the result is true for any abundant string of length $n \geq 5$, and let us prove it also holds for any abundant string of length $n+1$. Let S be a fully quaternary abundant string of length $n+1$. Because S is abundant, there exists a 1-flip in S leading to a string S' of length n. If S' is itself abundant, the induction hypothesis applies, and we are done since $d^s_{PS}(S) \leq 1 + (n-1)$. If not, by the above claim we know it is always possible to sort S' using at most $n-1$ reversals, and again we obtain $d^s_{PS}(S) \leq 1 + (n-1)$. □

Theorem 8. *For any constant integer $k \geq 4$, and for any fully k-ary string, the* SPSR *problem admits a PTAS.*

Proof. Let S be a fully k-ary string, where $k \geq 4$ is a constant. For any fixed $\epsilon > 0$ we set $N = (3k-4)/\epsilon + k$, and we distinguish two cases. First, let us suppose $n \geq N$. In proof of Proposition 2, an algorithm that groups S in at most $(n-3)$ reversals is given. Let us call the grouped string obtained this way S', and note that S' is of length k since we consider normalized strings. Now, it is not difficult to see that S' can be sorted in at most $2k-1$ reversals. Thus, altogether, we have an algorithm that sorts S and uses no more than $r(S) = (n-3) + (2k-1) = (n-k) + (3k-4)$ reversals. But, by Observation 1, we also know that $d^s_{PS}(S) \geq n - k$ (i). But since $n \geq N$, we have that $d^s_{PS}(S) \geq N - k$, that is $d^s_{PS}(S) \geq \frac{3k-4}{\epsilon} + k - k$, from which we conclude that $\epsilon d^s_{PS}(S) \geq 3k - 4$ (ii). Adding (i) and (ii) yields $(1+\epsilon)d^s_{PS}(S) \geq (n-k) + (3k-4)$, i.e. $r(S) \leq (1+\epsilon)d^s_{PS}(S)$, which proves the existence of a PTAS, since the underlying algorithm is clearly polynomial. Second, suppose that $N > n$. In that case, since ϵ and k are assumed to be constant, so is N. Hence, applying an exhaustive search from S yields the exact value of $d^s_{PS}(S)$. Let us call this algorithm $A'(S)$. At each iteration in $A'(S)$, there are at most $2n - 3 < 2N - 3$ possible reversals (as stated in proof of Theorem 6). Besides, since $d^s_{PS}(S) \leq 2n$, we conclude that it takes $O((2n-3)^{2n}) = O((2N-3)^{2N}) = O(1)$ time to execute $A'(S)$, which concludes the proof. □

5. Rearranging strings by prefix and suffix reversals

In this section, we describe our results concerning the RPSR problem. We start with the case where $n = k$, i.e. the case of permutations.

Theorem 9. *The* RPSR *problem is NP-hard for permutations.*

Proof. This result is obtained by a reduction from Sorting by Prefix Reversals in permutations, which has been proved to be NP-hard in [2]. More precisely, it is proved that it is NP-hard to decide whether a permutation π can be sorted using *exactly* $d_b(\pi)$ prefix reversals, where $d_b(\pi)$ is the number of breakpoints in π. Note that $d_b(\pi)$ is a lower bound on the number of p-reversals to sort π, since the number of breakpoints decreases by at most one each time a p-reversal is applied, and since

$d_b(I_n) = 0$. For any input permutation π of Sorting by Prefix Reversals, supposing π is built over the integers $1, 2 \dots n$, we create the permutation $\pi' = \pi.(n+1).(n+2)$. We now claim that π can be sorted using $d_b(\pi)$ p-reversals iff π' can be sorted using $d_b(\pi)$ p- and s-reversals.

($\Rightarrow$) Clearly, if π can be sorted using $d_b(\pi)$ p-reversals, this is also a valid scenario for p- and s-reversals, that also sorts π'.

($\Leftarrow$) Suppose π' can be sorted using by a sequence $s(\pi')$ of $d_b(\pi)$ p- and s-reversals. Because $d_b(\pi') = d_b(\pi)$, it can easily be seen that (i) no p-reversal ρ_p in $s(\pi')$ involves $(n+1)$ nor $(n+2)$ (otherwise, ρ_p would not decrease the number of breakpoints) and (ii) no s-reversal ρ_s is applied in $s(\pi')$ (otherwise, ρ_s would not decrease the number of breakpoints either). Hence, the sorting scenario for π' is also a valid p-reversals scenario for π, and it uses no more than $d_b(\pi)$ p-reversals. □

A straightforward consequence of the above theorem is that the SPSR is also NP-hard, since sorting and rearranging are equivalent when the input string is a permutation.

Corollary 2. *The* SPSR *problem is NP-hard for permutations.*

Combining Theorem 9 with Theorem 2, we get the following corollary.

Corollary 3. *The* RPSR *problem is NP-hard for strings built on any constant-size alphabet.*

The RPSR problem being NP-hard for small and big alphabets, one may look for approximation algorithms. We have the following result, which also applies to SPSR (see Corollary 4).

Proposition 9. *The* RPSR *problem admits an $O(1)$-approximation algorithm for any fully k-ary string of length n such that $n - k = O(1)$.*

Proof. The proof relies on a direct relationship between the RPSR problem and the Minimum Common String Partition (or MCSP) problem [11]. The MCSP problem takes as input two compatible strings S and T, and asks whether there exist a partition $\mathcal{S} = (S_1, S_2 \dots S_p)$ of S and a partition $\mathcal{T} = (T_1, T_2 \dots T_p)$ of T such that (i) for any S_i, there exists a T_j such that $S_i = T_j$ or $S_i = \overline{T_j}$, (ii) this mapping is bijective, and (iii) p is minimized. For any compatible strings S and T, let $p_{opt}(S, T)$ be the size of an optimal partition for MCSP. It can be seen that the following two inequalities hold:

1. $d^r_{PS}(S, T) \leq 3 \cdot p_{opt}(S, T)$: indeed, given an optimal partition of MCSP, one can transform S into T as follows: for j from $p_{opt}(S, T)$ down to 1, place T_j at its right location using only p-reversals. It suffices to note that each T_j can be correctly placed and oriented using at most 3 p-reversals to conclude.
2. $p_{opt}(S, T) \leq d^r_{PS}(S, T) + 1$. This comes from the fact that each p- or s-reversal cuts the initial string S in one location only. Hence, given an optimal p- and s-reversal rearrangement scenario, the places that have been cut determine a common string partition. There are at most $d^r_{PS}(S, T)$ cuts altogether, and thus $p_{opt}(S, T) \leq d^r_{PS}(S, T) + 1$.

For any string S, and for any letter $l \in \Sigma$, let us denote by $occ(S, l)$ the maximum number of occurrences of l in S, and let $occ(S) = \max_{l \in \Sigma}(occ(S, l))$ (note that if S and T are compatible, then $occ(S) = occ(T)$). It has been showed in [11] that MCSP is $O(occ(S))$-approximable, and by inequalities 1 and 2 above, we have that RSPR is $O(occ(S))$-approximable too. Now it suffices to note that when $n - k = c$, then $occ(S) \leq c + 1$. In particular, if $c = O(1)$, then $occ(S) = O(1)$, and RSPR is $O(1)$-approximable. □

If we replace T by sort(S) in proof of Proposition 9, we get the following result.

Corollary 4. *The* SPSR *problem admits an $O(1)$-approximation algorithm for any fully k-ary string of length n such that $n - k = O(1)$.*

We now finish this section by turning to the fixed-parameterized tractability of the three problems, starting with RPSR.

Theorem 10. *The* RPSR *problem is FPT, when parameterized by b_{max}.*

Proof. Theorem 3 in [1] shows that Sorting by Reversals in Strings is FPT when the parameter is b_{max}. We begin by a high-level description of its Proof. The proposed algorithm is based on a two-step procedure: first, "guess" in which blocks, or between which blocks, the extremities of the reversals take place. Such a description is called a *scaffold*, and generating all the possible scaffolds is exponential in b_{max} only, for the two following reasons:

1. the reversal distance is in $O(b_{max})$;
2. in any optimal reversal scenario between two strings S and T, any intermediate string contains $O(b_{max})$ blocks (Lemma 2 in [1]).

The second step of the procedure consists in checking, for each scaffold, whether it is valid (Lemma 3 in [1]). In other words, this second step determines (if it exists), for each reversal ρ, the precise positions of the extremities of ρ, which are so far only described at the block level by the scaffold. This is done using a network flow representation. The network order

and size being in $O((b_{max})^2)$, the computation of the flow is also polynomial in b_{max}, and the whole algorithm is thus FPT in b_{max}.

It can be seen that the above proof can be adapted to the RPSR problem. First, note that $d^r_{PS}(S, T) \leq 4b_{max}$. Indeed, grouping S (resp. T) into S_g (resp. T_g) takes no more than b_{max} reversals, and going from S_g to T_g takes at most $2k \leq 2b_{max}$ reversals (recall that k is the size of the alphabet). Second, in any optimal rearrangement scenario from S to T by p- and s-reversals, no intermediate string has strictly more than $3b_{max}$ blocks. If this was not the case, suppose S' is such a string. Let $b(S)$ (resp. $b(S')$) be the number of blocks in S (resp. S'). We would then have $d^r_{PS}(S, S') \geq b(S') - b(S) > 2b_{max}$, because a (prefix or suffix) reversal increases the number of blocks by at most 1. Similarly, $d^r_{PS}(S', T) > 2b_{max}$, and thus $d^r_{PS}(S', T) > 4b_{max}$, a contradiction to the fact that $d^r_{PS}(S, T) \leq 4b_{max}$, as previously mentioned. Third, a p-reversal (resp. s-reversal) cuts a string only in one place, and in the generation of all possible scaffolds (the "guessing" part), each guessed cut has to be combined with one out of two possibilities: either p- or s-reversal. As a conclusion, in our case, the total number of possible scaffolds remains exponential in b_{max} only. The second part of the algorithm, based on the network flow, can also be adapted to p- and s-reversals, and it is not hard to see that the order and size of each constructed network remain $O((b_{max})^2)$. □

Theorem 10 is adapted from [1]. Note also that it is valid regardless of the size of the alphabet. Another consequence is the following corollary; indeed, grouping and sorting are special cases of rearranging, in which the target string has $k = |\Sigma| \leq b_{max}$ blocks with $b_{max} = b(S)$.

Corollary 5. *The* GPSR *and* SPSR *problems are FPT, when parameterized by* $b(S)$.

6. Conclusion and open questions

In this paper, we have studied the three problems GPSR, SPSR and RPSR under an algorithmic viewpoint, and have provided a series of results, especially focused either on small-size alphabets ($k = O(1)$) and big alphabets ($k = n$, i.e. permutations, and more generally, $k = n - c$ with $c = O(1)$). Some questions remain open, and we would like to end this paper by listing a few of them: first, we showed in Section 2 classes of permutations for which a significant difference exists between the prefix and the prefix/suffix distance, for sorting and rearranging. What can be said in this context for the grouping problem? Second, so far our results do not contradict the fact that the grouping problem may be in P for any value $k = |\Sigma|$, and it is tempting to conjecture that it is indeed the case. Is this conjecture true? On the contrary, there seems to exist some border (in terms of k) between polynomiality and NP-hardness for the sorting problem: it is in P for small values of k, but NP-hard when $k = n$. Can we precisely detect that border? Concerning the rearranging problem, can it be approximated when $k = 2$? And what is the status concerning RPSR in terms of parameterized complexity, when the parameter is $b_{min}(S, T) = \min\{b(S), b(T)\}$? Finally, we note that the variant where strings are signed could also be studied.

References

[1] L. Bulteau, G. Fertin, C. Komusiewicz, (Prefix) reversal distance for (signed) strings with few blocks or small alphabets, J. Discrete Algorithms 37 (2016) 44–55.
[2] L. Bulteau, G. Fertin, I. Rusu, Pancake flipping is hard, J. Comput. System Sci. 81 (8) (2015) 1556–1574.
[3] A. Caprara, Sorting by reversals is difficult, in: Proceedings of the First Annual International Conference on Computational Molecular Biology, RECOMB '97, ACM, New York, NY, USA, 1997, pp. 75–83.
[4] D. Christie, Genome Rearrangement Problems(Ph.D. thesis), University of Glasgow, 1998.
[5] D.A. Christie, R.W. Irving, Sorting strings by reversals and by transpositions, SIAM J. Discret. Math. 14 (2) (2001) 193–206.
[6] G. Fertin, L. Jankowiak, G. Jean, Prefix and suffix reversals on strings, in: C.S. Iliopoulos, S.J. Puglisi, E. Yilmaz (Eds.), SPIRE 2015, in: Lecture Notes in Computer Science, vol. 9309, Springer, 2015, pp. 165–176.
[7] G. Fertin, A. Labarre, I. Rusu, E. Tannier, S. Vialette, Combinatorics of Genome Rearrangements, in: Computational Molecular Biology, MIT Press, 2009, p. 312.
[8] W.H. Gates, C.H. Papadimitriou, Bounds for sorting by prefix reversal, Discrete Math. 27 (1979) 47–57.
[9] S. Hannenhalli, P.A. Pevzner, Transforming cabbage into turnip: Polynomial algorithm for sorting signed permutations by reversals, J. ACM 46 (1) (1999) 1–27.
[10] C.A.J. Hurkens, L. van Iersel, J. Keijsper, S. Kelk, L. Stougie, J. Tromp, Prefix reversals on binary and ternary strings, SIAM J. Discrete Math. 21 (3) (2007) 592–611.
[11] P. Kolman, T. Waleń, Reversal distance for strings with duplicates: Linear time approximation using hitting set, Electron. J. Combin. 14 (1) (2007).
[12] C.N. Lintzmayer, G. Fertin, Z. Dias, Approximation algorithms for sorting by length-weighted prefix and suffix operations, Theoret. Comput. Sci. 593 (2015) 26–41.
[13] C.N. Lintzmayer, G. Fertin, Z. Dias, Sorting permutations by prefix and suffix rearrangements, J. Bioinforma. Comput. Biol. 15 (01) (2017) 1750002–1.
[14] A.J. Radcliffe, A.D. Scott, E.L. Wilmer, Reversals and transpositions over finite alphabets, SIAM J. Discret. Math (2005).
[15] M.K. Rahman, M.S. Rahman, Prefix and suffix transreversals on binary and ternary strings, J. Discrete Algorithms 33 (2015) 160–170.